SOFTWARE INGENIERITZA II
2026KO AZAROA

PROIEKTUAREN DISEINU PATROIAK

Aitzol Waliño eta Angela Noriega

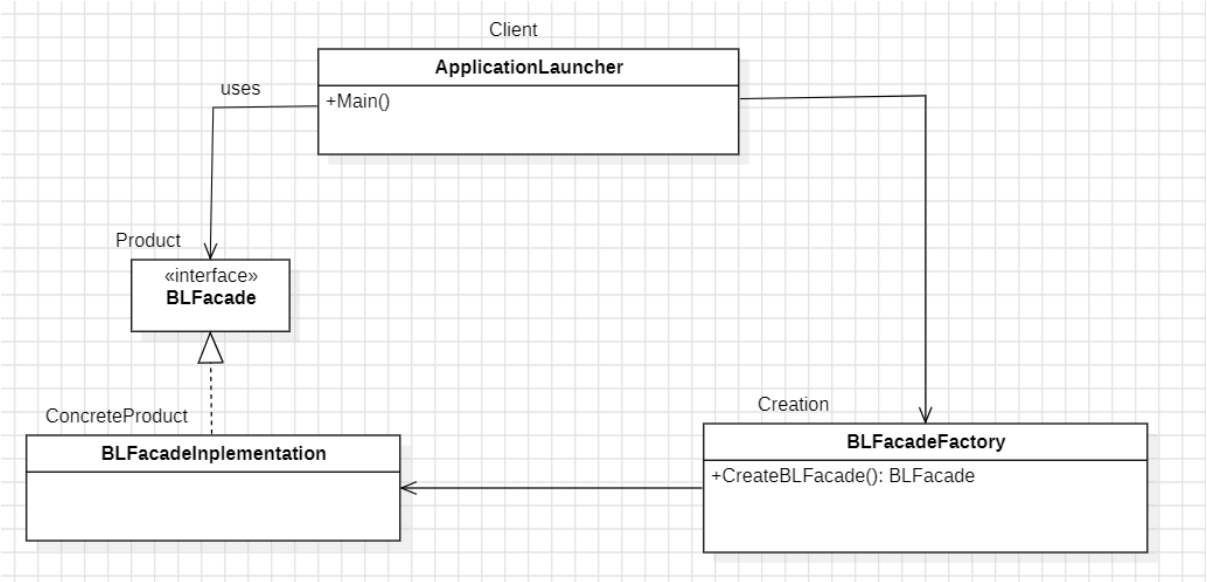


AURKIBIDEA

1-FACTORY METHOD PATROIA.....3
1-UML.....3
2-KODEA.....3
2-ITERATOR PATROIA..... 5
1-UML.....5
2-KODEA.....5
3-IRUDIA.....8
3-ADAPTER PATROIA..... 9
1-UML.....9
2-KODEA.....9
3-IRUDIA..... 11

1-FACTORY METHOD PATROIA

1-UML



2-KODEA

Lehenengo bertsioan, ApplicationLauncher klaseak zuzenean erabakitzen zuen zein negozio-logika objektu sortu behar zen. Horretarako, isBusinessLogicLocal() metodoaren emaitzaren arabera, BLFacadeImplementation objektu lokal bat edo web zerbitzuaren bitartez lortutako objektu urrun bat sortzen zuen.

```
try {
      BLFacade appFacadeInterface;
      UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
      if (c.isBusinessLogicLocal()) {

            DataAccess da= new DataAccess(c.getDataBaseOpenMode().equals("initialize"));
            appFacadeInterface=new BLFacadeImplementation(da);
      }
      else { //If remote

            String serviceName= "http://"+c.getBusinessLogicNode() +":"+ .getBusinessLogic ort()+"/ws/"+c.getBusinessLogicName()+"?w
            URL url = new URL(serviceName);
            QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
            Service service = Service.create(url, qname);

               appFacadeInterface = service.getPort(BLFacade.class);
            }
            MainGUI.setBussinessLogic(appFacadeInterface);

      }catch (Exception e) {
            a.jLabelSelectOption.setText("Error: "+e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);

            System.out.println("Error in ApplicationLauncher: "+e.toString());
      }
}
```


Horrek esan nahi zuen ApplicationLauncher klaseak negozio-logikaren inplementazio zehatzak ezagutu behar zituela, eta objektuen sorrera bertan zegoela zentralizatuta.

Horregatik, sortu dugun BLFacadeFactory klasean zentralizatu dugu objektuen sorrera. Horrela, ApplicationLauncher klaseak ez du jakin behar zein inplementazio erabiltzen den, faktoriari BLFacade objektu bat eskatzea besterik ez du egiten.

ApplicationLauncher:

```
try {
    UIManager.setLookAndFeel("avax.swing.plaf.metal.MetalLookAndFeel");
    BLFacade appFacadeInterface =BLFacadeFactory.createBLFacade();
    MainGUI.setBussinessLogic(appFacadeInterface);
```

BLFacadeFactory:

```
public class BLFacadeFactory {

   public static BLFacade createBLFacade() throws Exception {

       ConfigXML c = ConfigXML.getInstance();

       if (c.isBusinessLogicLocal()) {

           DataAccess da = new DataAccess();
           return new BLFacadeImplementation(da);

       } else {

           String serviceName = "http://"
                   + c.getBusinessLogicNode()
                   + ":"
                   + c.getBusinessLogicPort()
                   + "/ws/"
                   + c.getBusinessLogicName()
                   + "?wsdl";

           URL url = new URL(serviceName);

           QName qname =new QName("http://businessLogic/","BLFacadeImplementationService");

           Service service = Service.create(url, qname);

           return service.getPort(BLFacade.class);
       }
   }
}
```

Ondorioz, objektuen sorrera zentralizatu da, aurkezpen geruza eta negozio-logika gutxiago lotuta daude, inplementazio berriak gehitzea errazagoa da, Mockito edo mock objektuak erabiltzea errazagoa da eta faktorian beste BLFacade inplementazio bat gehi daiteke aurkezpenaren kodea aldatu gabe.

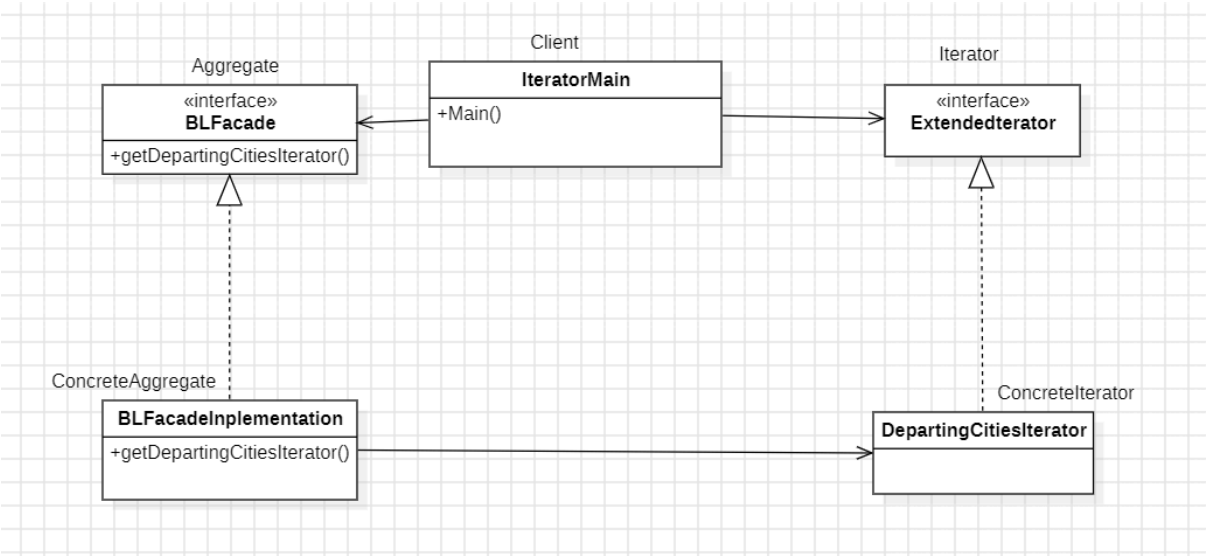
Patroiaren rolak

- Client: ApplicationLauncher

- Product: BLFacade
- ConcreteProduct: BLFacadeImplementation eta urruneko BLFacade inplementazioa
- Creator (Factory): BLFacadeFactory

2-ITERATOR PATROIA

1-UML



2-KODEA

Iterator metodoa inplementatzeko, lehenik Iterator paketea sortu dugu erabiliko ditugun klase berriak gordetzeko. ExtendedIterator interfazea sortu dugu Java-ko Iterator interfazea hedatuz.

```
public interface ExtendedIterator<Object> extends Iterator<Object> {
    //return the actual element and go to the previous
    public Object previous();
    //true if ther is a previous element
    public boolean hasPrevious();
    //It is placed in the first element
    public void goFirst();
//      It is placed in the last element
    public void goLast();
    }
```

Interfaze honek Iterator eskaintzen dituen aukera guztiak ematen ditu, hortaz aparte elementuak aurrerantz zein atzerantz zeharkatzeko aukera ematen du, baita lehen edo azken elementuan kokatzeko ere.

Interfazea erabiltzeko lehenik eta behin, DepartingCitiesIterator klasea sortu da, ExtendedIterator<String> interfazea inplementatuz. Klase honek irteerako hirien zerrenda gordetzen du eta posizio-indize baten bidez zerrendan zehar nabigatzea ahalbidetzen du.

```
public class DepartingCitiesIterator implements ExtendedIterator<String>{
            private List<String> cities;
             private int pos;
             public DepartingCitiesIterator(List<String> cities) {
                this.cities = cities;
                this.pos = 0;
             }
             @Override
             public boolean hasNext() {
                return pos < cities.size();
             }
             @Override
             public String next() {
                return cities.get(pos++);
             }
             @Override
             public boolean hasPrevious() {
                return pos > 0;
             }
             @Override
             public String previous() {
                return cities.get(--pos);
             }
             @Override
             public void goFirst() {
                pos = 0;
             }
             @Override
             public void goLast() {
                    pos = cities.size();
             }
}
```

Gainera, BLFacade interfazean metodo berria gehitu da:

```
    @WebMethod public ExtendedIterator<String> getDepartingCitiesIterator();
```

Metodo hori BLFacadeImplementation klasean inplementatu da. Bertan, irteerako hiri guztiak lortzen dira eta horien gainean iteradore bat sortzen da:

```
@WebMethod public ExtendedIterator<String> getDepartingCitiesIterator() {

    List<String> cities = getDepartCities();

    return new DepartingCitiesIterator(cities);
}
```

Iteradoreari esker, bezeroak ez du hirien zerrendaren barne-egitura ezagutu behar. Nabigazioa iteradorearen bidez egiten da.

Azkenik, Main bat sortu dugu iteradorea probatzeko.

```
public class IteratorMain {

    public static void main(String[] args) {

        try {
            BLFacade blFacade = BLFacadeFactory.createBLFacade();

            ExtendedIterator<String> i =blFacade.getDepartingCitiesIterator();

            String c;
            System.out.println("_____________________");
            System.out.println("FROM LAST TO FIRST");
            i.goLast();
            while (i.hasPrevious()) {
                c = i.previous();
                System.out.println(c);
            }

            System.out.println();
            System.out.println("_____________________");
            System.out.println("FROM FIRST TO LAST");

            i.goFirst();

            while (i.hasNext()) {
                c = i.next();
                System.out.println(c);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

Patroiaren rolak

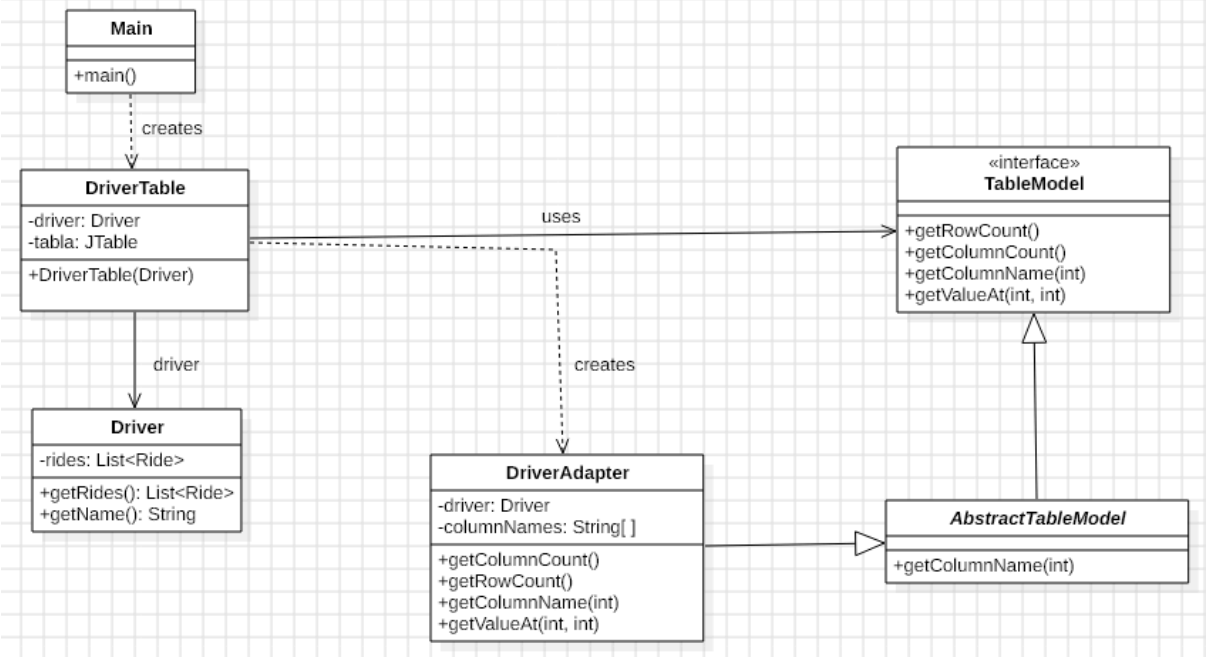
- Iterator: ExtendedIterator
- ConcreteIterator: DepartingCitiesIterator
- Aggregate: irteerako hirien bilduma (List<String>)
- Client: IteratorMain

3-IRUDIA



3-ADAPTER PATROIA

1-UML



2-KODEA

JTable-ak TableModel interfazea betetzen duen eredu bat behar du datuak erakusteko. Hala ere, Driver klaseak ez du interfaze hori betetzen, bere bidaiak List<Ride> zerrenda batean gordetzen baititu. Horregatik, DriverAdapter klasea sortu dugu, AbstractTableModel-etik heredatzen duena eta Driver objektu bat jasotzen duena. Honek Driver-en interfazea JTable-ak espero duenera egokitzen du, Driver klasea aldatu gabe.

DiverAdapter:

```java
public class DriverAdapter extends AbstractTableModel {

    private static final long serialVersionUID = 1L;

    private final Driver driver;
    private final String[] columnNames = { "from", "to", "date", "places", "price" };

    public DriverAdapter(Driver driver) {
        this.driver = driver;
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public int getRowCount() {
        return driver.getRides().size();
    }

    @Override
    public String getColumnName(int col) {
        return columnNames[col];
    }

    @Override
    public Object getValueAt(int row, int col) {
        List<Ride> rides = driver.getRides();
        Ride ride = rides.get(row);
        switch (col) {
        case 0:
            return ride.getFrom();
        case 1:
            return ride.getTo();
        case 2:
            return ride.getDate();
        case 3:
            return ride.getnPlaces();
        case 4:
            return ride.getPrice();
        default:
            return null;
        }
    }
}
```

Metodo garrantzitsuena getValueAt da, taulako zutabe bakoitza Ride baten eremu batekin lotzen baitu (from, to, date, nPlaces, price).

DriverTable klaseak DriverAdapter sortzen du eta JTable-ari pasatzen dio. Horrela, JTable-ak ez du Driver ezagutu behar, TableModel interfazea bakarrik.

DriverTable:

```java
DriverAdapter adapt = new DriverAdapter(driver);
tabla = new JTable(adapt);
```

Main-ean, BLFacadeFactory-ren bidez negozio-logika lortzen da, “Urtzi” gidaria bilatzen da eta leihoa bistaratzen da. Horretarako getDriver(String izena) metodoa gehitu dugu BLFacade-n, BLFacadeImplementation-en eta DataAccess-en, eta “Urtzi” gidaria bere lau bidaiekin initializeDB()-n sartu dugu.

```
public class Main {

    public static void main(String[] args) throws Exception {
        BLFacade blFacade = BLFacadeFactory.createBLFacade();
        Driver d = blFacade.getDriver("Urtzi");
        DriverTable dt = new DriverTable(d);
        dt.setVisible(true);
    }
}
```

```
public Driver getDriver(String izena) {
    open();
    TypedQuery<Driver> query = db.createQuery("SELECT d FROM Driver d WHERE d.izena=?1", Driver.class);
    query.setParameter(1, izena);
    List<Driver> gidariak = query.getResultList();
    close();
    return gidariak.isEmpty() ? null : gidariak.get(0);
}
```

```
Driver driver4=new Driver("driver4@gmail.com","Urtzi");
```

```
driver4.addRide("Donostia", "Madrid", UtilDate.newDate(year,month,30), 5, 20);
driver4.addRide("Irun", "Donostia", UtilDate.newDate(year,month,30), 5, 2);
driver4.addRide("Madrid", "Donostia", UtilDate.newDate(year,month,10), 5, 5);
driver4.addRide("Barcelona", "Madrid", UtilDate.newDate(year,month,20), 0, 10);
```

3-IRUDIA

Urtzi's rides

from	to	date	places	price
Donostia	Madrid	Mon Nov 30 00:00:00 CET 2026	5	20.0
Irun	Donostia	Mon Nov 30 00:00:00 CET 2026	5	2.0
Madrid	Donostia	Tue Nov 10 00:00:00 CET 2026	5	5.0
Barcelona	Madrid	Fri Nov 20 00:00:00 CET 2026	0	10.0